Dossier Chenillard

Matières	Chenillard
État	Terminé

Maël ENDRUSCHAT g1B

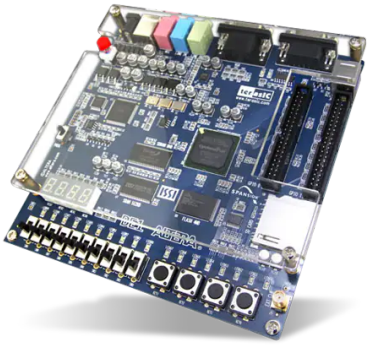
Robin JALADE

Institut Universitaire de Technologie Aix-Marseille Université

Sommaire.

Introduction.

L'objectif de cette SAé est de mener une conception d'une solution logicielle permettant l'animation lumineuse sur une carte ALTERA en conformité avec le cahier des charges et effectuer les tests de vérification du chenillard numérique réalisé.



Carte ALTERA

Cahier des charges.

Concevoir un programme d'animation lumineuse sur une carte d'évaluation QUARTUS à l'aide du langage de description VHDL.
Le chenillard à réaliser devra allumer successivement des leds et/ou des afficheurs selon le mode sélectionné et la vitesse de défilement choisie.

Il possédera :

- Différents modes de fonctionnement sélectionnable par des switchs de la carte ;
- Une vitesse de défilement variable et réglable par des switchs de la carte ;
- Une fonction "pause" à l'aide d'un bouton poussoir de la carte ;
- Une fonction "marche/arrêt" à l'aide d'un switch de la carte ;
- Une fonction "réinitialisation" à l'aide d'un bouton poussoir de la carte.

Spécifications :

SPECIFICATIONS FONCTIONNELLES

1. Allumer selon le mode de fonctionnement sélectionné :

   Les LEDs rouges

   Et/ou les LEDs vertes

   Et/ou les segments des afficheurs de la carte ALTERA DE1
2. Mettre en marche ou en pause
3. Réinitialiser le cycle de fonctionnement
4. Régler la vitesse de défilement

SPECIFICATIONS OPERATIONNELLES

- **4 vitesses de défilement :** 10Hz, 5Hz, 2Hz et 1Hz. Choix de la vitesse avec les switchs SW(0) et SW(1)
- **9 modes de fonctionnement sélectionnés avec les switchs SW(2), SW(3), SW(4), SW(5) :**
  - **Mode 1 :** les LEDs rouges s'allument successivement de la 0 à la 7, puis cela recommence de 0 à 7…
  - **Mode 2 :** les LEDs rouges s'allument successivement de la 0 à la 7, puis de la 7 à la 0, puis à nouveau de la 0 à la 7…
  - **Mode 3 :** les LEDs rouges s'allument de la 0 à la 7 pendant que les leds vertes s'allument de la 7 à la 0 et le cycle recommence.
  - **Mode 4 :** les LEDs rouges suivent le mode 2, pendant que les LEDs vertes s'allument de la 7 à la 0 puis de la 0 à la 7…
  - **Mode 5 :** les segments de l'afficheur HEX0 s'allument successivement, du segment a jusqu'au segment g…
  - **Mode 6 :** les segments des 4 afficheurs s'allument en même temps, segments a puis segments b,…
  - **Mode 7 :** les afficheurs s'allument les uns après les autres en suivant le cycle segment a puis b, puis c…
  - **Mode 8 :** les afficheurs clignotent en affichant 2024
  - **Mode 9 :** les modes 4 et 8 s'ajoutent
  - **Mode 10 :** mode original (ex. : défilement d'un texte)

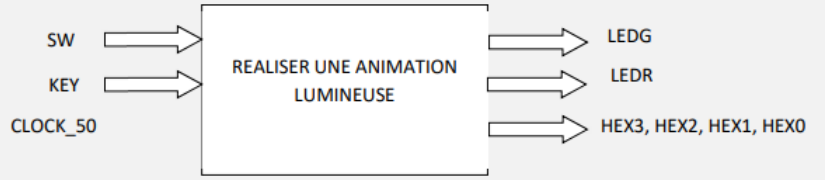
- **RAZ :** le bouton poussoir KEY(2) permet de réinitialiser le mode de défilement c'est-à-dire de reprendre le défilement à partir de sa position initiale
- **PAUSE :** le bouton poussoir KEY(0) permet de mettre en pause le système (l'allumage s'arrête sur la position actuelle).
- **M/A :** le switch SW(9) permet de mettre en marche ou d'arrêter le système (à l'arrêt toutes les LEDs et les afficheurs sont éteints).

SPECIFICATIONS TECHNOLOGIQUES

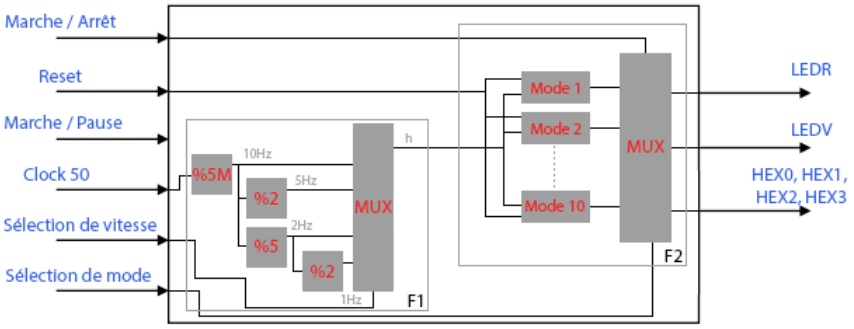
- Carte ALTERA DE1
- Logiciel QUARTUS

Description fonctionnelle.

Le schéma fonctionnel d'ordre 1 :



Le schéma fonctionnel d'ordre 2 :



Réalisation.

METHODES DE REALISATION : VHDL + OUTILS GRAPHIQUE

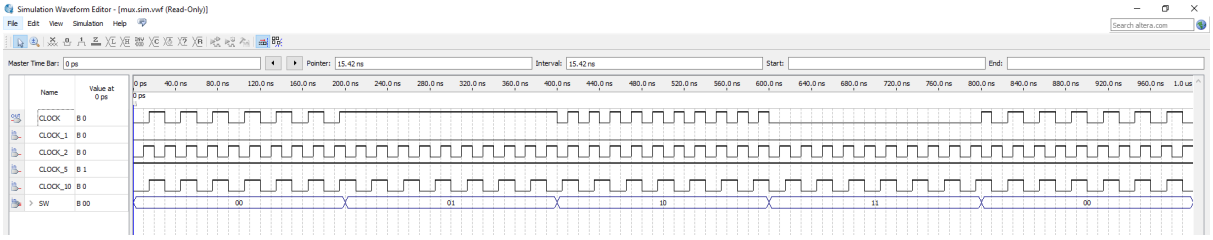
- Nous avons écrit les fichiers VHDL de chacun des blocs ci-dessus (diviseur, mux...).
- Nous avons validé individuellement chacun des blocs.
- Nous avons créé le symbole des différents blocs.
- Et nous avons assemblé les différents blocs dans un fichier graphique pour former nos fonctions que nous avons reliées ensemble.

PROCEDURE DE VALIDATION :

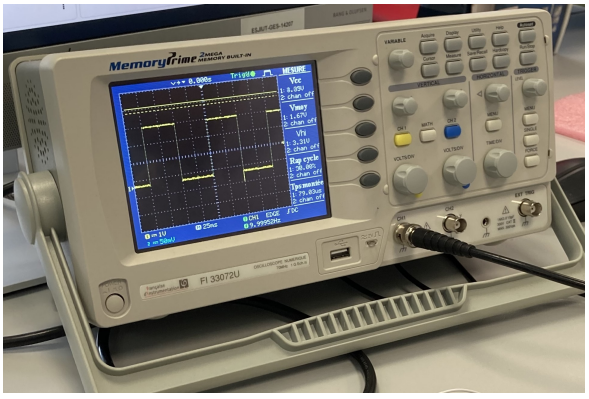
1. Dans un premier temps, n'ayant pas accès à une carte de test, nous avons utilisé la simulation intégrée au logiciel Quartus.
2. Ensuite, nous avons vérifié sur la carte ALTERA si le programme fonctionnait comme nous le souhaitions.
3. Parallèlement, nous avons visualisé sur oscillogramme pour confirmer le bon fonctionnement.

Rapport des procédures des tests fonctionnels et opérationnels.

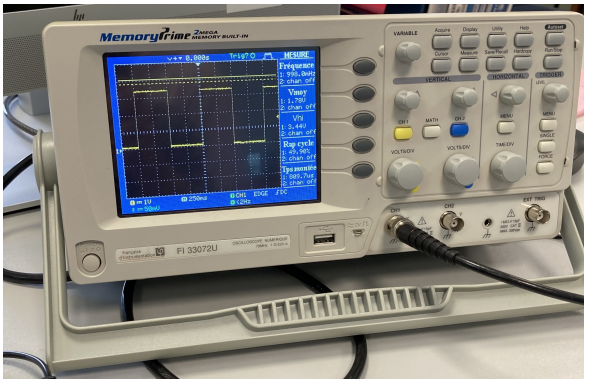
Simulation de la fonction 1 :



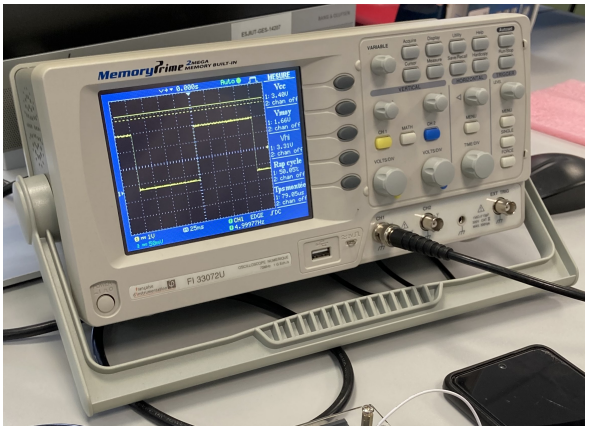
Vérification des diviseurs sur oscilloscope :



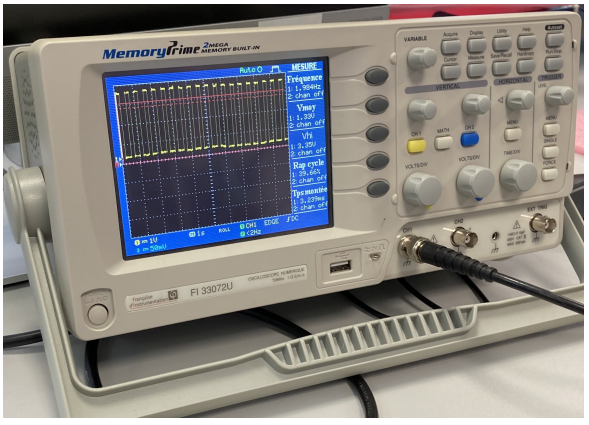
10Hz



1Hz



5Hz



2Hz

Programme VHDL commenté.

Fonction 1.

Diviseur 5M :

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;

entity diviseur_10 IS
    PORT(
        LEDR : out std_logic_vector(0 downto 0);
        CLOCK_50 : in std_logic;
        GPIO_0 : out std_logic_vector(0 downto 0)
    );
end diviseur_10;

architecture diviseur of diviseur_10 IS
signal r : std_logic_vector(22 downto 0);
signal s_clock_10 : std_logic := '0';
begin
    process(CLOCK_50)
    begin

        if rising_edge(CLOCK_50) then
            if r = 4999999 then r <= "00000000000000000000000";
                else r <= r+1;
            end if;

            if r < 2500000 then  s_clock_10 <= '1';
                else s_clock_10 <= '0';
            end if;
            LEDR(0) <= s_clock_10;
        end if;
        GPIO_0(0) <= s_clock_10;
    end process;

end diviseur;
```

Diviseur 2 :

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;

entity diviseur_5 IS
PORT(
    LEDR : out std_logic_vector(1 downto 1);
    CLOCK_10 : in std_logic;
```

```
        GPIO_0 : out std_logic_vector(1 downto 1)
);
end diviseur_5;

architecture diviseur of diviseur_5 IS
signal s_clock_5 : std_logic := '0';
begin
    process(CLOCK_10)
    begin
        if rising_edge(CLOCK_10) then
            s_clock_5 <= not s_clock_5;
        end if;
        LEDR(1) <= s_clock_5;
    end process;
     GPIO_0(1) <= s_clock_5;

end diviseur;
```

Diviseur 5 :

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;

entity diviseur_2 IS
    PORT(
        LEDR : out std_logic_vector(2 downto 2);
        CLOCK_10 : in std_logic;
              GPIO_0 : out std_logic_vector(2 downto 2)
    );
end diviseur_2;

architecture diviseur of diviseur_2 IS
signal r : std_logic_vector(2 downto 0);
signal s_clock_2 : std_logic := '0';
begin
    process(CLOCK_10)
    begin
        if rising_edge(CLOCK_10) then
            if r = 4 then r <= "000";
                else r <= r+1;
            end if;

            if r < 2 then  s_clock_2 <= '1';
                else s_clock_2 <= '0';
            end if;
            LEDR(2) <= s_clock_2 ;
        end if;
          GPIO_0(2) <= s_clock_2;
    end process;

end diviseur;
```

Diviseur 2 :

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;

entity diviseur_1 IS
    PORT(
        LEDR : out std_logic_vector(3 downto 3);
        CLOCK_2 : in std_logic;
              GPIO_0 : out std_logic_vector(3 downto 3)
    );
end diviseur_1;

architecture diviseur of diviseur_1 IS
signal r : std_logic_vector(1 downto 0);
signal s_clock_1 : std_logic := '0';
begin
    process(CLOCK_2)
    begin
        if rising_edge(CLOCK_2) then
            if r = 1 then r <= "00";
                else r <= r+1;
            end if;

            if r < 1 then  s_clock_1 <= '1';
                else s_clock_1 <= '0';
            end if;
            LEDR(3) <= s_clock_1 ;
        end if;
          GPIO_0(3) <= s_clock_1;
    end process;

end diviseur;
```

Mux :

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

entity mux is
    Port (
        CLOCK_10 : in std_logic;
        CLOCK_5 : in std_logic;
        CLOCK_2 : in std_logic;
        CLOCK_1 : in std_logic;
        SW : in std_logic_vector(1 downto 0);
        CLOCK : buffer std_logic
    );
    end mux;
    architecture fonc_mux of mux is
    begin
        CLOCK <= CLOCK_10 when SW = "00" else
        CLOCK_5 when SW = "01" else
```

```
        CLOCK_2 when SW = "10" else
        CLOCK_1;
    end fonc_mux;
```

Fonction 2.

Mode 1 :

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;

ENTITY mode_1 IS
PORT (
    CLOCK : in std_logic;
    Reset : in std_logic_vector(2 downto 2);
    LEDR : buffer std_logic_vector (7 downto 0);
    q : buffer STD_LOGIC_VECTOR(2 DOWNTO 0)
);
END mode_1;

ARCHITECTURE mode of mode_1 is
BEGIN
PROCESS (CLOCK)
    BEGIN
        IF rising_edge(CLOCK) THEN
            IF Reset = "0" THEN q <= "000";
                ELse IF q = 7 THEN q <= "000";
                    ELSE q <= q + 1;
                END IF;
            END IF;
        END IF;
END PROCESS;
process(CLOCK)
    begin
        if rising_edge(CLOCK) then
            case q is
                when "000" => LEDR <= "00000001";
                when "001" => LEDR <= "00000010";
                when "010" => LEDR <= "00000100";
                when "011" => LEDR <= "00001000";
                when "100" => LEDR <= "00010000";
                when "101" => LEDR <= "00100000";
                when "110" => LEDR <= "01000000";
                when "111" => LEDR <= "10000000";
                when others => LEDR <= "00000000";
            end case;
        end if;
end process;
END mode;
```

Mode 2 :

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;

ENTITY mode_2 IS
PORT (
        CLOCK : in std_logic;
    Reset : in std_logic_vector(2 downto 2);
    LEDR : buffer std_logic_vector (7 downto 0);
        q : buffer STD_LOGIC_VECTOR(3 DOWNTO 0)
);
END mode_2;

ARCHITECTURE mode of mode_2 is
BEGIN
process(CLOCK)
begin
    if rising_edge(CLOCK) then
        IF Reset = "0" THEN q <= "0000";
            ELse if q = 14 then q <= "0000";
                else q <= q + 1;
            end if;
            end if;
    end if;
end process;
process(CLOCK)
begin
    if rising_edge(CLOCK) then
        case q is
            when "0000" => LEDR <= "00000001";
            when "0001" => LEDR <= "00000010";
            when "0010" => LEDR <= "00000100";
            when "0011" => LEDR <= "00001000";
            when "0100" => LEDR <= "00010000";
            when "0101" => LEDR <= "00100000";
            when "0110" => LEDR <= "01000000";
            when "0111" => LEDR <= "10000000";
            when "1000" => LEDR <= "01000000";
            when "1001" => LEDR <= "00100000";
            when "1010" => LEDR <= "00010000";
            when "1011" => LEDR <= "00001000";
            when "1100" => LEDR <= "00000100";
            when "1101" => LEDR <= "00000010";
            when "1110" => LEDR <= "00000001";
            when others => LEDR <= "00000000";
        end case;
    end if;
end process;
END mode;
```

Mode 3 :

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;

ENTITY mode_3 IS
PORT (
        CLOCK : in std_logic;
    Reset : in std_logic_vector(2 downto 2);
    LEDR : buffer std_logic_vector (7 downto 0);
    LEDG : buffer std_logic_vector (7 downto 0);
        q : buffer STD_LOGIC_VECTOR(2 DOWNTO 0)
);
END mode_3;

ARCHITECTURE mode of mode_3 is
BEGIN
process(CLOCK)
begin
    if rising_edge(CLOCK) then
        IF Reset = "0" THEN q <= "000";
            Else if q = 7 then q <= "000";
                else q <= q + 1;
            end if;
            end if;
    end if;
end process;
process(CLOCK)
begin
    if rising_edge(CLOCK) then
        case q is
            when "000" => LEDR <= "00000001";
            when "001" => LEDR <= "00000010";
            when "010" => LEDR <= "00000100";
            when "011" => LEDR <= "00001000";
            when "100" => LEDR <= "00010000";
            when "101" => LEDR <= "00100000";
            when "110" => LEDR <= "01000000";
            when "111" => LEDR <= "10000000";
            when others => LEDR <= "00000000";
        end case;
    end if;
end process;
process(CLOCK)
begin
    if rising_edge(CLOCK) then
        case q is
            when "000" => LEDG <= "10000000";
            when "001" => LEDG <= "01000000";
            when "010" => LEDG <= "00100000";
            when "011" => LEDG <= "00010000";
            when "100" => LEDG <= "00001000";
            when "101" => LEDG <= "00000100";
            when "110" => LEDG <= "00000010";
```

```
            when "111" => LEDG <= "00000001";
            when others => LEDG <= "00000000";
        end case;
    end if;
end process;
END mode;
```

Mode 4 :

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;

ENTITY mode_4 IS
PORT (
        CLOCK : in std_logic;
    Reset : in std_logic_vector(2 downto 2);
    LEDR : buffer std_logic_vector (7 downto 0);
    LEDG : buffer std_logic_vector (7 downto 0);
        q : buffer STD_LOGIC_VECTOR(3 DOWNTO 0)
);
END mode_4;

ARCHITECTURE mode of mode_4 is
BEGIN
process(CLOCK)
begin
    if rising_edge(CLOCK) then
        IF Reset = "0" THEN q <= "0000";
            Else if q = 14 then q <= "0000";
                else q <= q + 1;
            end if;
            end if;
    end if;
end process;
process(CLOCK)
begin
    if rising_edge(CLOCK) then
        case q is
            when "0000" => LEDR <= "00000001";
            when "0001" => LEDR <= "00000010";
            when "0010" => LEDR <= "00000100";
            when "0011" => LEDR <= "00001000";
            when "0100" => LEDR <= "00010000";
            when "0101" => LEDR <= "00100000";
            when "0110" => LEDR <= "01000000";
            when "0111" => LEDR <= "10000000";
            when "1000" => LEDR <= "01000000";
            when "1001" => LEDR <= "00100000";
            when "1010" => LEDR <= "00010000";
            when "1011" => LEDR <= "00001000";
            when "1100" => LEDR <= "00000100";
            when "1101" => LEDR <= "00000010";
            when "1110" => LEDR <= "00000001";
```

```
            when others => LEDR <= "00000000";
        end case;
    end if;
end process;
process(CLOCK)
begin
    if rising_edge(CLOCK) then
        case q is
            when "0000" => LEDG <= "00000001";
            when "0001" => LEDG <= "00000010";
            when "0010" => LEDG <= "00000100";
            when "0011" => LEDG <= "00001000";
            when "0100" => LEDG <= "00010000";
            when "0101" => LEDG <= "00100000";
            when "0110" => LEDG <= "01000000";
            when "0111" => LEDG <= "10000000";
            when "1000" => LEDG <= "01000000";
            when "1001" => LEDG <= "00100000";
            when "1010" => LEDG <= "00010000";
            when "1011" => LEDG <= "00001000";
            when "1100" => LEDG <= "00000100";
            when "1101" => LEDG <= "00000010";
            when "1110" => LEDG <= "00000001";
            when others => LEDG <= "00000000";
        end case;
    end if;
end process;
END mode;
```

Mode 5 :

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;

ENTITY mode_5 IS
PORT (
    CLOCK : in std_logic;
    Reset : in std_logic_vector(2 downto 2);
    HEX0 : buffer std_logic_vector (6 downto 0);
    q : buffer STD_LOGIC_VECTOR(2 DOWNTO 0)
);
END mode_5;

ARCHITECTURE mode of mode_5 is
BEGIN
PROCESS (CLOCK)
BEGIN
    IF rising_edge(CLOCK) THEN
        IF Reset = "0" THEN q <= "000";
            ELse IF q = 7 THEN q <= "000";
                ELSE q <= q + 1;
```

```
            END IF;
        END IF;
        END IF;
END PROCESS;

process(CLOCK)
    begin
        if rising_edge(CLOCK) then
            case q is
                when "000" => HEX0 <= "1111110";
                when "001" => HEX0 <= "1111101";
                when "010" => HEX0 <= "1111011";
                when "011" => HEX0 <= "1110111";
                when "100" => HEX0 <= "1101111";
                when "101" => HEX0 <= "1011111";
                when "110" => HEX0 <= "0111111";
                when others => HEX0 <= "1111111";
            end case;
        end if;
end process;
END mode;
```

Mode 6 :

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;

ENTITY mode_6 IS
PORT (
    CLOCK : in std_logic;
  Reset : in std_logic_vector(2 downto 2);
    HEX0 : buffer std_logic_vector (6 downto 0);
    HEX1 : buffer std_logic_vector (6 downto 0);
    HEX2 : buffer std_logic_vector (6 downto 0);
    HEX3 : buffer std_logic_vector (6 downto 0);
    q : buffer STD_LOGIC_VECTOR(2 DOWNTO 0)
);
END mode_6;

ARCHITECTURE mode of mode_6 is
BEGIN
PROCESS (CLOCK)
BEGIN
    IF rising_edge(CLOCK) THEN
        IF Reset = "0" THEN q <= "000";
            ELsIF q = 7 THEN q <= "000";
                ELSE q <= q + 1;
            END IF;
        END IF;
END PROCESS;
process(CLOCK)
    begin
        if rising_edge(CLOCK) then
```

```
            CASE q IS
                WHEN "000" =>
                    HEX1 <= "1111110";
                    HEX3 <= "1111110";
                    HEX2 <= "1111110";
                    HEX0 <= "1111110";
                WHEN "001" =>
                    HEX1 <= "1111101";
                    HEX3 <= "1111101";
                    HEX2 <= "1111101";
                    HEX0 <= "1111101";
                WHEN "010" =>
                    HEX1 <= "1111011";
                    HEX3 <= "1111011";
                    HEX2 <= "1111011";
                    HEX0 <= "1111011";
                WHEN "011" =>
                    HEX1 <= "1110111";
                    HEX3 <= "1110111";
                    HEX2 <= "1110111";
                    HEX0 <= "1110111";
                WHEN "100" =>
                    HEX1 <= "1101111";
                    HEX3 <= "1101111";
                    HEX2 <= "1101111";
                    HEX0 <= "1101111";
                WHEN "101" =>
                    HEX1 <= "1011111";
                    HEX3 <= "1011111";
                    HEX2 <= "1011111";
                    HEX0 <= "1011111";
                WHEN "110" =>
                    HEX1 <= "0111111";
                    HEX3 <= "0111111";
                    HEX2 <= "0111111";
                    HEX0 <= "0111111";
                WHEN OTHERS =>
                    HEX1 <= "1111111";
                    HEX3 <= "1111111";
                    HEX2 <= "1111111";
                    HEX0 <= "1111111";
            END CASE;
        end if;
end process;
END mode;
```

Mode 7 :

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;

ENTITY mode_7 IS
PORT (
```

```
    CLOCK : in std_logic;
  Reset : in std_logic_vector(2 downto 2);
    HEX0 : buffer std_logic_vector (6 downto 0);
    HEX1 : buffer std_logic_vector (6 downto 0);
    HEX2 : buffer std_logic_vector (6 downto 0);
    HEX3 : buffer std_logic_vector (6 downto 0);
    q : buffer STD_LOGIC_VECTOR(4 DOWNTO 0)
);
END mode_7;

ARCHITECTURE mode of mode_7 is
BEGIN
PROCESS (CLOCK)
BEGIN
    IF rising_edge(CLOCK) THEN
        IF Reset = "0" THEN q <= "00000";
            ELsIF q = 27 THEN q <= "00000";
                ELSE q <= q + 1;
            END IF;
        END IF;
END PROCESS;
process(CLOCK)
    begin
        if rising_edge(CLOCK) then
            CASE q IS
                WHEN "00000" =>
                    HEX1 <= "1111111";
                                        HEX2 <= "1111111";
                                        HEX3 <= "1111111";
                      HEX0 <= "1111110";

                WHEN "00001" =>
                    HEX0 <= "1111101";

                WHEN "00010" =>
                    HEX0 <= "1111011";

                WHEN "00011" =>
                    HEX0 <= "1110111";

                WHEN "00100" =>
                    HEX0 <= "1101111";

                WHEN "00101" =>
                    HEX0 <= "1011111";

                WHEN "00110" =>
                    HEX0 <= "0111111";

                                WHEN "00111" =>
                                        HEX0 <= "1111111";
                    HEX1 <= "1111110";

                WHEN "01000" =>
                    HEX1 <= "1111101";
```

```
 WHEN "01001" =>
     HEX1 <= "1111011";

 WHEN "01010" =>
     HEX1 <= "1110111";

 WHEN "01011" =>
     HEX1 <= "1101111";

 WHEN "01100" =>
     HEX1 <= "1011111";

 WHEN "01101" =>
     HEX1 <= "0111111";

WHEN "01110" =>
    HEX1 <= "1111111";
    HEX2 <= "1111110";

 WHEN "01111" =>
     HEX2 <= "1111101";

 WHEN "10000" =>
     HEX2 <= "1111011";

 WHEN "10001" =>
     HEX2 <= "1110111";

 WHEN "10010" =>
     HEX2 <= "1101111";

 WHEN "10011" =>
     HEX2 <= "1011111";

 WHEN "10100" =>
     HEX2 <= "0111111";

 WHEN "10101" =>
     HEX2 <= "1111111";
     HEX3 <= "1111110";

 WHEN "10110" =>
     HEX3 <= "1111101";

 WHEN "10111" =>
     HEX3 <= "1111011";

 WHEN "11000" =>
     HEX3 <= "1110111";

 WHEN "11001" =>
     HEX3 <= "1101111";

 WHEN "11010" =>
```

```
                    HEX3 <= "1011111";

                WHEN "11011" =>
                    HEX3 <= "0111111";

                WHEN OTHERS =>
                    HEX0 <= "1111111";
                                    HEX1 <= "1111111";
                                    HEX2 <= "1111111";
                                    HEX3 <= "1111111";
            END CASE;
        end if;
end process;
END mode;
```

Mode 8 :

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;

ENTITY mode_8 IS
    PORT (
        CLOCK : in std_logic;
            Reset : in std_logic_vector(2 downto 2);
        HEX0 : BUFFER STD_LOGIC_VECTOR (6 DOWNTO 0);
        HEX1 : BUFFER STD_LOGIC_VECTOR (6 DOWNTO 0);
        HEX2 : BUFFER STD_LOGIC_VECTOR (6 DOWNTO 0);
        HEX3 : BUFFER STD_LOGIC_VECTOR (6 DOWNTO 0)
    );
END mode_8;

ARCHITECTURE mode OF mode_8 IS
    SIGNAL q : INTEGER RANGE 0 TO 1 := 0;
    SIGNAL affichage_2024 : BOOLEAN := TRUE;
BEGIN
    PROCESS (CLOCK)
    BEGIN
        IF rising_edge(CLOCK) THEN q <= q + 1;
          IF Reset = "0" THEN q <= 0;
            ELSE IF q = 1 THEN q <= 0;
                affichage_2024 <= NOT affichage_2024;
            END IF;
            END IF;
            IF affichage_2024 THEN
                HEX0 <= "0011001"; -- 4
                HEX1 <= "0100100"; -- 2
                HEX2 <= "1000000"; -- 0
                HEX3 <= "0100100"; -- 2
            ELSE
                HEX0 <= "1111111";
                HEX1 <= "1111111";
                HEX2 <= "1111111";
                HEX3 <= "1111111";
```

```
            END IF;
        END IF;
    END PROCESS;
END mode;
```

Mode 10 :

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;

ENTITY mode_10 IS
PORT (
    CLOCK : in std_logic;
  Reset : in std_logic_vector(2 downto 2);
    HEX0 : buffer std_logic_vector (6 downto 0);
    HEX1 : buffer std_logic_vector (6 downto 0);
    HEX2 : buffer std_logic_vector (6 downto 0);
    HEX3 : buffer std_logic_vector (6 downto 0);
    q : buffer STD_LOGIC_VECTOR(4 DOWNTO 0)
);
END mode_10;

ARCHITECTURE mode of mode_10 is
BEGIN
PROCESS (CLOCK)
BEGIN
    IF rising_edge(CLOCK) THEN
        IF Reset = "0" THEN q <= "00000";
            Else IF q = 27 THEN q <= "00000";
                ELSE q <= q + 1;
            END IF;
        END IF;
    END IF;
END PROCESS;
process(CLOCK)
    begin
        if rising_edge(CLOCK) then
            CASE q IS
                WHEN "00000" =>
                    HEX0 <= "1111110";

                WHEN "00001" =>
                    HEX0 <= "1111101";

                WHEN "00010" =>
                    HEX0 <= "1111011";

                WHEN "00011" =>
                    HEX0 <= "1110111";

                WHEN "00100" =>
                    HEX0 <= "1101111";
```

```
WHEN "00101" =>
    HEX0 <= "1011111";

WHEN "00110" =>
    HEX0 <= "0111111";

                WHEN "00111" =>
    HEX1 <= "1111110";

WHEN "01000" =>
    HEX1 <= "1111101";

WHEN "01001" =>
    HEX1 <= "1111011";

WHEN "01010" =>
    HEX1 <= "1110111";

WHEN "01011" =>
    HEX1 <= "1101111";

WHEN "01100" =>
    HEX1 <= "1011111";

WHEN "01101" =>
    HEX1 <= "0111111";

                WHEN "01110" =>
    HEX2 <= "1111110";

WHEN "01111" =>
    HEX2 <= "1111101";

WHEN "10000" =>
    HEX2 <= "1111011";

WHEN "10001" =>
    HEX2 <= "1110111";

WHEN "10010" =>
    HEX2 <= "1101111";

WHEN "10011" =>
    HEX2 <= "1011111";

WHEN "10100" =>
    HEX2 <= "0111111";

WHEN "10101" =>
    HEX3 <= "1111110";

WHEN "10110" =>
    HEX3 <= "1111101";
```

```
                WHEN "10111" =>
                    HEX3 <= "1111011";

                WHEN "11000" =>
                    HEX3 <= "1110111";

                WHEN "11001" =>
                    HEX3 <= "1101111";

                WHEN "11010" =>
                    HEX3 <= "1011111";

                WHEN "11011" =>
                    HEX3 <= "0111111";

                WHEN OTHERS =>
                    HEX0 <= "1111111";
                                    HEX1 <= "1111111";
                                    HEX2 <= "1111111";
                                    HEX3 <= "1111111";
            END CASE;
        end if;
end process;
END mode;
```

Mux :

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

entity mux_out is
    Port (
      LEDR_1 : in std_logic_vector(7 downto 0);

      LEDR_2 : in std_logic_vector(7 downto 0);

      LEDR_3 : in std_logic_vector(7 downto 0);
            LEDG_3 : in std_logic_vector(7 downto 0);

      LEDR_4 : in std_logic_vector(7 downto 0);
            LEDG_4 : in std_logic_vector(7 downto 0);

      HEX0_5 : in std_logic_vector(6 downto 0);

      HEX0_6 : in std_logic_vector(6 downto 0);
            HEX1_6 : in std_logic_vector(6 downto 0);
            HEX2_6 : in std_logic_vector(6 downto 0);
            HEX3_6 : in std_logic_vector(6 downto 0);

      HEX0_7 : in std_logic_vector(6 downto 0);
            HEX1_7 : in std_logic_vector(6 downto 0);
            HEX2_7 : in std_logic_vector(6 downto 0);
            HEX3_7 : in std_logic_vector(6 downto 0);

      HEX0_8 : in std_logic_vector(6 downto 0);
```

```
            HEX1_8 : in std_logic_vector(6 downto 0);
            HEX2_8 : in std_logic_vector(6 downto 0);
            HEX3_8 : in std_logic_vector(6 downto 0);

            HEX0_10 : in std_logic_vector(6 downto 0);
            HEX1_10 : in std_logic_vector(6 downto 0);
            HEX2_10 : in std_logic_vector(6 downto 0);
            HEX3_10 : in std_logic_vector(6 downto 0);


            HEX0 : buffer std_logic_vector (6 downto 0);
            HEX1 : buffer std_logic_vector (6 downto 0);
            HEX2 : buffer std_logic_vector (6 downto 0);
            HEX3 : buffer std_logic_vector (6 downto 0);

        LEDR : out std_logic_vector(7 downto 0);
            LEDG : out std_logic_vector(7 downto 0);

            Arret : in std_logic_vector(9 downto 9);
            SW : in std_logic_vector(5 downto 2)
    );
    end mux_out;
    architecture fonc_mux of mux_out is
    begin
        LEDR <= "00000000" when Arret = "0" else
                LEDR_1 when SW = "0000" else
        LEDR_2 when SW = "0001" else
        LEDR_3 when SW = "0010" else
        LEDR_4 when SW = "0011" else
                LEDR_4 when SW = "1000" else
                "00000000";

        LEDG <= "00000000" when Arret = "0" else
                LEDG_3 when SW = "0010" else
        LEDG_4 when SW = "0011" else
                LEDG_4 when SW = "1000" else
                "00000000";

        HEX0 <= "1111111" when Arret = "0" else
                HEX0_5 when SW = "0100" else
        HEX0_6 when SW = "0101" else
        HEX0_7 when SW = "0110" else
        HEX0_8 when SW = "0111" else
                HEX0_8 when SW = "1000" else
                HEX0_10 when SW = "1001" else
                "1111111";

        HEX1 <= "1111111" when Arret = "0" else
                HEX1_6 when SW = "0101" else
        HEX1_7 when SW = "0110" else
        HEX1_8 when SW = "0111" else
                HEX1_8 when SW = "1000" else
                HEX1_10 when SW = "1001" else
                "1111111";
```

```
        HEX2 <= "1111111" when Arret = "0" else
                HEX2_6 when SW = "0101" else
        HEX2_7 when SW = "0110" else
        HEX2_8 when SW = "0111" else
                HEX2_8 when SW = "1000" else
                HEX2_10 when SW = "1001" else
                "1111111";

        HEX3 <= "1111111" when Arret = "0" else
                HEX3_6 when SW = "0101" else
        HEX3_7 when SW = "0110" else
        HEX3_8 when SW = "0111" else
                HEX3_8 when SW = "1000" else
                HEX3_10 when SW = "1001" else
                "1111111";
    end fonc_mux;
```

Conclusion.

Bilan technique :

Le cahier des charges a été suivi à la lettre, toutes les fonctionnalités requises ont été implémentées. Nous avons utilisé les boîtes à outils Quartus et VHDL des TPs de SIN.

Bilan personnel :

Ce projet nous a donné l'opportunité de concevoir un produit exigé par un client, avec un cahier des charges strict à respecter dans un délai imparti. Nous devions utiliser le langage précis VHDL et une carte ALTERA.